

Python para Solemne 3 - Cheat Sheet

Pandas: leer e inspeccionar

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv(url)
```

```
print(df.head())
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
```

<code>pd.read_csv(url)</code>	lee el CSV y crea el DataFrame
<code>df.head()</code>	muestra las primeras filas
<code>df.shape</code>	entrega filas y columnas
<code>df.dtypes</code>	muestra tipos de datos
<code>df.isna().sum()</code>	cuenta faltantes por columna

Pandas: limpiar datos

```
cols = ["col_1", "col_2", "col_3"]
```

```
df_limpio = df.dropna(subset=cols).copy()
```

<code>cols = [...]</code>	lista de columnas clave
---------------------------	-------------------------

<code>dropna(subset=cols)</code>	borra filas incompletas solo en esas columnas
----------------------------------	---

<code>copy()</code>	crea una tabla independiente
---------------------	------------------------------

<code>len(df_limpio)</code>	cuenta filas después de limpiar
-----------------------------	---------------------------------

Pandas: columnas y grupos

```
df_limpio["nueva_col"] = (
    df_limpio["col_a"]**2 *
    (df_limpio["col_b"] / valor_ref)**4
)
```

```
grupo = df_limpio.groupby("col_categoria")
```

<code>df["nueva_col"] = ...</code>	crea o reemplaza una columna
<code>df["col_a"]**2</code>	eleva una columna al cuadrado
<code>(...)</code>	permite partir una operación larga
<code>groupby("col")</code>	agrupa filas por categoría
<code>grupo = ...</code>	guarda el grupo para usarlo después

Pandas: resumen por grupo

```
df_num = df_limpio.drop(columns=["col_texto"]).copy()
grupo_num = df_num.groupby("col_categoria")
```

```
cantidad = grupo_num["col_categoria"].count()
promedios = grupo_num.mean()
desv = grupo_num.std()
```

<code>drop(columns=[...])</code>	quita columnas de texto
<code>grupo_num["col"].count()</code>	cuenta datos por grupo
<code>grupo_num.mean()</code>	promedio por grupo
<code>grupo_num.std()</code>	desviación por grupo

Listas y contadores

```
datos = [valor_1, valor_2, valor_3]
copia = datos.copy()
```

```
primero = copia[0]
ultimo = copia[-1]
```

```
contador = 0
contador = contador + 1
```

```
return resultado, contador_1, contador_2
```

<code>datos = [...]</code>	crea una lista
<code>datos.copy()</code>	copia una lista sin cambiar la original
<code>copia[0]</code>	primer elemento
<code>copia[-1]</code>	último elemento
<code>contador = contador + 1</code>	aumenta una cuenta
<code>return a, b, c</code>	devuelve tres valores

Condiciones y mensajes

```
if valor == -1:
    print("no encontrado")
else:
    print("encontrado:", valor)
```

```
if len(lista) > 0:
    print(lista[0])
```

<code>if condicion:</code>	ejecuta si la condición se cumple
<code>else:</code>	ejecuta el caso alternativo
<code>len(lista) > 0</code>	verifica que exista un elemento
<code>print(...)</code>	muestra texto o variables

Python para Solemne 3 - Cheat Sheet

NumPy: arreglos y modelo

```
x = np.array([x0, x1, x2])
y = np.array([y0, y1, y2])
```

```
x_fino = np.linspace(xmin, xmax, 200)
```

```
a, b, c = np.polyfit(x, y, deg=2)
y_modelo = a*x**2 + b*x + c
```

`np.array([...])` crea un arreglo numérico

`np.linspace(a,b,n)` crea una grilla entre a y b

`np.polyfit(x,y,deg=2)` ajusta un polinomio cuadrático

`a, b, c = ...` guarda coeficientes

`a*x**2 + b*x + c` evalúa el modelo

Cálculos con arreglos

```
diferencia = y - y_modelo
suma = np.sum(diferencia**2)
promedio = np.mean(y)
```

`y - y_modelo` resta elemento a elemento

`diferencia**2` cuadrado de cada diferencia

`np.sum(...)` suma todos los valores

`np.mean(y)` calcula el promedio

Raíces con np.roots

```
# Resolver a*x**2 + b*x + c = 0
raices = np.roots([a, b, c])
```

```
# Resolver a*x**2 + b*x + c = umbral
raices = np.roots([a, b, c - umbral])
```

`np.roots([...])` calcula raíces de un polinomio

`[a, b, c]` entrega coeficientes desde mayor a menor grado

`c - umbral` mueve el umbral al lado izquierdo de la ecuación

`raices` puede contener valores reales o complejos

Filtrar raíces

```
soluciones = []

for r in raices:
    es_real = abs(r.imag) < tol
    esta_en_rango = xmin <= r.real <= xmax
```

```
    if es_real and esta_en_rango:
        soluciones.append(r.real)
```

`for r in raices:` recorre una raíz a la vez

`r.imag` parte imaginaria

`r.real` parte real

`abs(r.imag) < tol` verifica que sea casi real

`lista.append(valor)` agrega una solución

Imprimir resultados

```
print("valor:", valor)
print(f"valor = {valor:.3f}")
```

```
if len(soluciones) > 0:
    print(soluciones[0])
else:
    print("sin solución")
```

`print("texto", x)` imprime texto y variable

`f"...{x:.3f}"` muestra tres decimales

`len(soluciones)` cuenta soluciones
`soluciones[0]` toma la primera solución

Operadores y errores

`and` une condiciones normales de Python

`&` une condiciones dentro de filtros de pandas

`|` acepta una condición u otra en pandas

`(...)` separa condiciones y operaciones largas

`np.roots([c,b,a])` orden incorrecto de coeficientes

`tomar toda raíz` revise si es real y está en rango